

图论

Graph

Rosaya

Oasis-Rosaya

目录

1. 树基础	2
1.1 定义	3
1.2 树的重心	10
1.3 dfs 序、括号序和欧拉序	13
1.4 LCA	16
1.5 直径和中心	21
1.6 习题	24
2. 最小生成树	25
2.1 定义	26
2.2 Kruskal	27
2.3 Prim	30
2.4 习题	34

目录

1. 树基础	2
1.1 定义	3
1.2 树的重心	10
1.3 dfs 序、括号序和欧拉序	13
1.4 LCA	16
1.5 直径和中心	21
1.6 习题	24
2. 最小生成树	25
2.1 定义	26
2.2 Kruskal	27
2.3 Prim	30
2.4 习题	34

1.1 定义

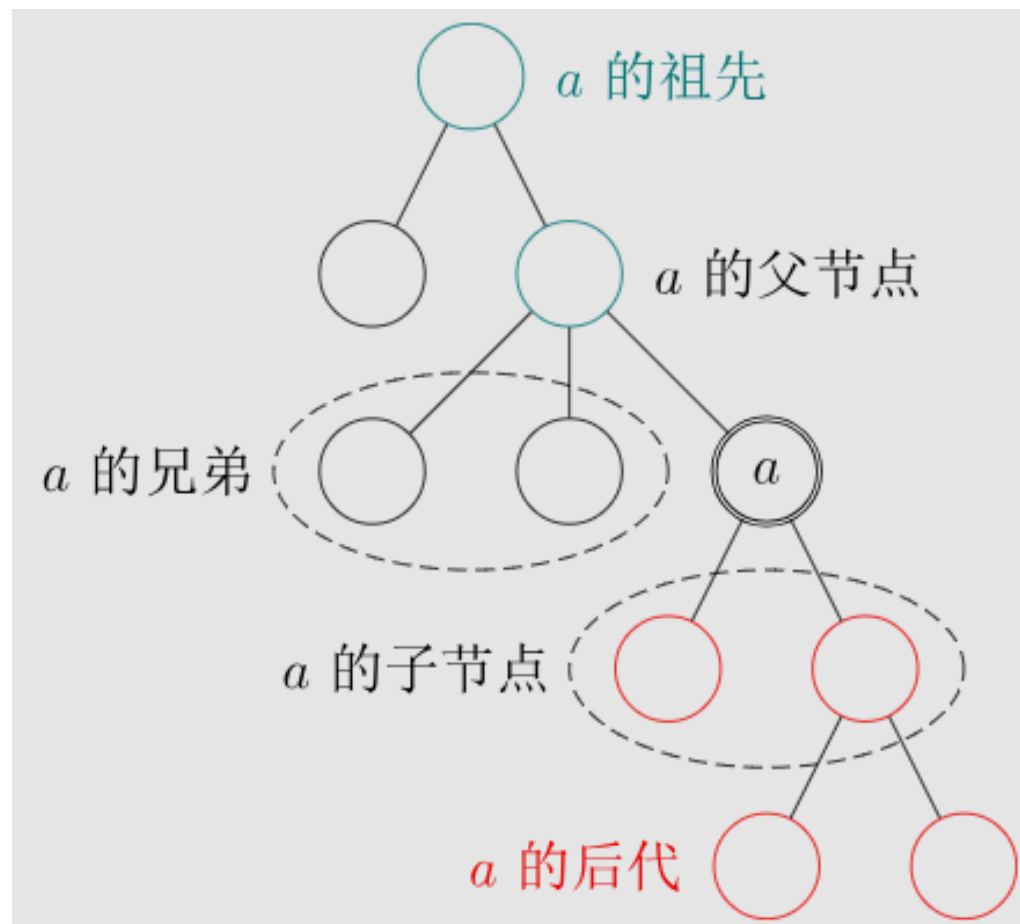
1.1 定义

一个没有固定根结点的树称为**无根树** (unrooted tree)。无根树有几种等价的形式化定义：

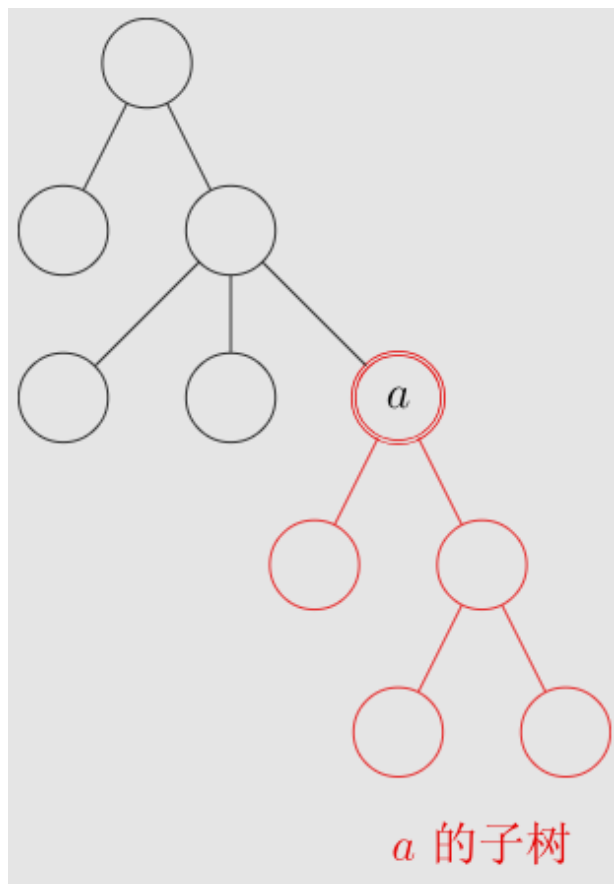
- (1) 有 n 个结点， $n - 1$ 条边的连通无向图。
- (2) 无向无环的连通图。
- (3) 任意两个结点之间有且仅有一条简单路径的无向图。
- (4) 任何边均为桥的连通图。
- (5) 没有圈，且在任意不同两点间添加一条边之后所得图含唯一的一个圈的图。

在无根树的基础上，指定一个结点称为**根**，则形成一棵**有根树** (rooted tree)。有根树在很多时候仍以无向图表示，只是规定了结点之间的上下级关系。后面的定义基于无向图。

1.1 定义



1.1 定义



1.1 定义

- 生成树：一个连通无向图的生成子图，同时要求是树。在图的边集中选择 $n - 1$ 条，将所有顶点连通。
- 无根树的叶结点：度数不超过 1 的结点（在有根树中，一般定义为没有子结点的结点）。
- k 叉树：所有点子结点个数不超过 k 的树。特别地，二叉树分左右儿子。
- 距离：两点之间的最短路径长度。
- 深度：与根的距离。
- 链：满足与任一结点相连的边不超过 2 条的树称为链。
- 菊花：满足存在 u 使得所有除 u 以外结点均与 u 相连的树称为菊花。

1.1 定义

• 广义菊花计数

定义一个 n 个点的有标号树是**广义菊花**，当且仅当其能找到一个根，满足：

- 记根深度为 0，那么所有点的深度均不超过 2。
- 深度为 1 的点的个数不少于 $\lfloor \frac{n}{2} \rfloor$ 。
- 以深度为 1 的任意一点为根的子树大小均不超过 $\lceil \frac{n}{4} \rceil$ 。

对 P 取模。

$$3 \leq n \leq 5000, 10^8 \leq P \leq 10^9 + 9.$$

1.1 定义

- $n \leq 500$

1.1 定义

- $n \leq 500$

对于有根树计数来说，其实就是一个选择父亲的过程。

记 $f_{i,j}$ 表示当前一共选择 i 个点且有 j 个点深度为 1 的树的数量，初始 $f_{1,0} = 1$ 。

转移时枚举新选择的子树的大小，然后再剩余 $n - i$ 个中选择 k 个，再选一个作为根（方案数 $k \binom{n-i}{k}$ ），需要注意这样会重复计数，所以我们强制选择**剩余未选点中编号最小的那个**。

$$f_{i+k,j+1} \leftarrow f_{i,j} \times k \binom{n-i-1}{k-1}。$$

复杂度 $O(n^3)$ 。

1.1 定义

- 正解

正难则反，考虑容斥。

由于对每个子树都有大小限制，我们不妨看看如果某棵子树违反了 $\frac{n}{4}$ 的大小限制会发生什么。

此时深度 ≤ 1 的点有至少 $\frac{n}{2} + 1$ 个，而违反限制的子树至少有 $\frac{n}{4}$ 个深度为 2。

也就是说剩余深度为 2 的结点已经不超过 $\frac{n}{4} - 1$ 个了，无论如何都无法再违反限制。

也就是说我们先不考虑第三条要求，然后枚举一个子树大小违反限制容斥即可。

复杂度 $O(n^2)$ 。

1.2 树的重心

1.2 树的重心

树的**重心** x 有以下几个等价定义：

- (1) $\sum_{i=1}^n \text{dist}(i, x)$ 最小。
- (2) 删除 x 分为若干个子树，子树节点数的最大值最小。
- (3) 以 x 为根，所有子树大小均 $\leq \frac{n}{2}$ 。

1.2 树的重心

树的**重心** x 有以下几个等价定义：

- (1) $\sum_{i=1}^n \text{dist}(i, x)$ 最小。
- (2) 删除 x 分为若干个子树，子树节点数的最大值最小。
- (3) 以 x 为根，所有子树大小均 $\leq \frac{n}{2}$ 。

有以下性质：

- (1) 至多有两个重心，且如果有两个重心，则他们相邻。
- (2) 将两棵树用一条边连成一棵新的树，则新树重心在原来两棵树的路径上。
- (3) 在树上添加或删除一个叶子，重心至多只移动一条边的距离。由此我们还可以引申出**加权重心**的定义。

1.2 树的重心

- 树上建小学

给定一棵树，边有边权， i 号点上有 a_i 个小朋友。

现在打算在某个点建立小学，使得所有孩子到小学的总距离最短。

也就是 $\sum_{i=1}^n a_i \text{dist}(i, x)$ 最小。

$1 \leq n \leq 3 \times 10^5$, $0 \leq a_i \leq 10^3$ 。

1.2 树的重心

• 树上建小学

给定一棵树，边有边权， i 号点上有 a_i 个小朋友。

现在打算在某个点建立小学，使得所有孩子到小学的总距离最短。

也就是 $\sum_{i=1}^n a_i \text{dist}(i, x)$ 最小。

$1 \leq n \leq 3 \times 10^5$, $0 \leq a_i \leq 10^3$ 。

根据重心性质的证明，可以先任找一个点 u 当作根，然后找到加权子树大小 s_v 最大的子树，判定其是否有总和 $S = \sum a_i$ 的一半。

1.2 树的重心

• 树上建小学

给定一棵树，边有边权， i 号点上有 a_i 个小朋友。

现在打算在某个点建立小学，使得所有孩子到小学的总距离最短。

也就是 $\sum_{i=1}^n a_i \text{dist}(i, x)$ 最小。

$1 \leq n \leq 3 \times 10^5$, $0 \leq a_i \leq 10^3$ 。

根据重心性质的证明，可以先任找一个点 u 当作根，然后找到加权子树大小 s_v 最大的子树，判定其是否有总和 $S = \sum a_i$ 的一半。

因为可以注意到加权距离其实只跟左右两侧的加权子树大小相关，所以可以按求重心的方法来求加权重心。

1.2 树的重心

- **CF685B Kay and Snowflake**

给定一棵大小为 n 的有根树，求出每一棵子树（有根树意义下且包含整颗树本身）的重心是哪一个节点。

$1 \leq n \leq 3 \times 10^5$ 。

1.2 树的重心

- CF685B Kay and Snowflake

给定一棵大小为 n 的有根树，求出每一棵子树（有根树意义下且包含整颗树本身）的重心是哪一个节点。

$$1 \leq n \leq 3 \times 10^5。$$

对于一棵以点 u 为根的子树，其重心一定在所有以 u 为父亲的直接子节点为根的子树（进一步的，应该是**最大的子树**）的重心到点 u 的路径上。

1.2 树的重心

- CF685B Kay and Snowflake

给定一棵大小为 n 的有根树，求出每一棵子树（有根树意义下且包含整颗树本身）的重心是哪一个节点。

$$1 \leq n \leq 3 \times 10^5。$$

对于一棵以点 u 为根的子树，其重心一定在所有以 u 为父亲的直接子节点为根的子树（进一步的，应该是**最大的子树**）的重心到点 u 的路径上。

对于每棵以节点 u 为根的子树，先求出所有以其直接子节点为根的子树的重心（叶子节点的重心是其本身），然后从最大子树的重心向上判断路径上的节点是不是重心即可。

时间复杂度为 $O(n)$ 可以求出所有子树的重心。

1.3 dfs 序、括号序和欧拉序

1.3 dfs 序、括号序和欧拉序

对有根树进行 dfs，并按以下方式生成序列：

dfs 序：每个点在被访问时加入。（长度为 n ）

括号序：每个点在被访问时和离开时加入。（长度为 $2n$ ）

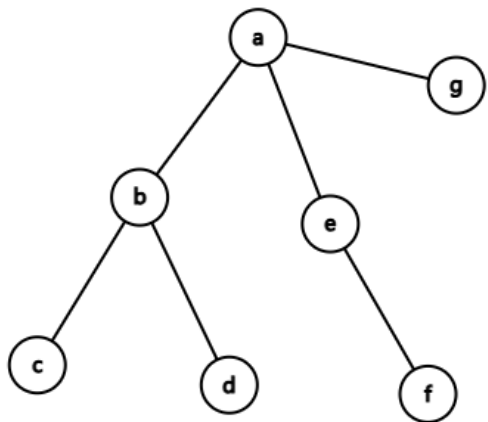
欧拉序：每个点在被访问和访问完每个儿子时加入。（长度为 $2n - 1$ ）

1.3 dfs 序、括号序和欧拉序

```
1 void dfs(int u,int fa){
2     dfn[++dft]=u; // dfsnum, dfstime
3     for(int v:adj[u]){
4         if(v!=fa){
5             dfs(v,u); // 欧拉序
6         }
7     } // 括号序
8 }
```



1.3 dfs 序、括号序和欧拉序



dfs 序: abcdefg

括号序: abccddbef fegga

欧拉序: abcbdbae f eaga

优势: 子树占据连续一段区间。

1.4 LCA

1.4 LCA

- **P3379 【模板】最近公共祖先 (LCA)**

给定一个大小为 n 的树， T 次询问，每次求 a, b 的最近公共祖先 (LCA)。

最近公共祖先就是字面意思，深度最大的公共祖先。

$1 \leq n, T \leq 5 \times 10^5$ 。

1.4 LCA

可以从 a, b 两个点开始一步一步跳父亲（每次跳深度较大的），这样可以 $O(n)$ 求出最近公共祖先。

1.4 LCA

可以从 a, b 两个点开始一步一步跳父亲（每次跳深度较大的），这样可以 $O(n)$ 求出最近公共祖先。

我们定义 k 级祖先就是沿着父边跳 k 次走到的点（如果深度不够就认为跳到一个虚空节点，一般认为是 0）。

首先我们不妨令 $\text{dep}_a \geq \text{dep}_b$ ，那么我们先让 a 跳到和 b 深度相同的位置，也就是 a 的 $\text{dep}_a - \text{dep}_b$ 级祖先，此时再判断 a, b 是否相同。

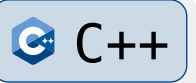
若相同则无事发生，否则我们二分 LCA 的高度 h ，检查 h 级祖先是否相同。

我们发现可以通过倍增的方式完成，我们预处理所有点的 2^k 级祖先，然后可以在时间 $O(\log n)$ 的复杂度内完成。

LCA 一般用于快速找到路径。

1.4 LCA

```
1  int LCA(int u, int v){
2      if(d[u]<d[v]) swap(u, v);
3      for(int i=20; i>=0; i--){
4          if(d[anc[u][i]]>=d[v]){
5              u=anc[u][i];
6          }
7      }
```



1.4 LCA

```
1    if(u==v) return u;
2    for(int i=20;i>=0;i--){
3        if(anc[u][i]!=anc[v][i]){
4            u=anc[u][i];
5            v=anc[v][i];
6        }
7    }
8    return anc[u][0];
9 }
```



1.4 LCA

- 最近公共祖先 **ex**

给定一棵大小为 n 的树， T 次询问，求 $[a, b]$ 之间的所有点的最近公共祖先。

$1 \leq n \leq 5 \times 10^5$, $1 \leq T \leq 5 \times 10^6$ 。

1.4 LCA

- 最近公共祖先 **ex**

给定一棵大小为 n 的树， T 次询问，求 $[a, b]$ 之间的所有点的最近公共祖先。

$1 \leq n \leq 5 \times 10^5$, $1 \leq T \leq 5 \times 10^6$ 。

考虑一个更好刻画 LCA 的方式——欧拉序。

若要求 a, b 的 LCA，假设 a, b 在欧拉序中第一次出现的位置是 x, y ，那么 a, b 的 LCA 就是欧拉序 $x \sim y$ 位中深度最小的点，可以用 st 表维护。

1.4 LCA

- 最近公共祖先 ex

给定一棵大小为 n 的树， T 次询问，求 $[a, b]$ 之间的所有点的最近公共祖先。

$1 \leq n \leq 5 \times 10^5$, $1 \leq T \leq 5 \times 10^6$ 。

考虑一个更好刻画 LCA 的方式——欧拉序。

若要求 a, b 的 LCA，假设 a, b 在欧拉序中第一次出现的位置是 x, y ，那么 a, b 的 LCA 就是欧拉序 $x \sim y$ 位中深度最小的点，可以用 st 表维护。

如何证明？提示： a, b 路径上的所有点都出现在 LCA 的子树中。

这样我们发现区间 LCA 也只用找区间中在欧拉序出现最早和最晚的点，然后查询欧拉序区间深度最小的点即可，复杂度 $O(\log n)$ 。

1.5 直径和中心

1.5 直径和中心

树上任意两节点之间最长的简单路径即为树的**直径**。直径可能有很多条。

- (1) 任取树上一点 x ，距离 x 最远的一个点必定出现在某条直径上。
- (2) 若存在多条直径，存在一点 x 使得所有直径均经过该点。
- (3) 若合并两棵树，则新树的直径端点一定是旧树的直径端点。

1.5 直径和中心

树上任意两节点之间最长的简单路径即为树的**直径**。直径可能有很多条。

- (1) 任取树上一点 x ，距离 x 最远的一个点必定出现在某条直径上。
- (2) 若存在多条直径，存在一点 x 使得所有直径均经过该点。
- (3) 若合并两棵树，则新树的直径端点一定是旧树的直径端点。

在树中，如果节点 x 作为根节点时，深度最大的点深度最小，那么称 x 为这棵树的**中心**。

- (1) 中心最多有两个，且是直径的中点。
- (2) 树上所有点到其最远点的路径一定交会于树的中心。
- (3) 当通过在两棵树间连一条边以合并为一棵树时，连接两棵树的中心可以使新树的直径最小。

这样直径可以通过两遍 dfs 求出，中心同理。

1.5 直径和中心

- 搞搞新意思

给定一棵 n 个点边带权的树, m 次询问, 每次可以任选一条边把边权改成 w_i (可以不改), 求最短直径, 询问独立。

$$1 \leq n, m \leq 10^5, \quad 1 \leq w \leq 10^9。$$

1.5 直径和中心

首先观察到若修改,一定修改的是原来直径上的边,否则原来直径不变且最大,肯定不改。

否则考虑修改直径第 i 条边的贡献。有跨过该边的路径和不跨过的,假设该边是 $p \rightarrow w$, 原来直径是 d 。

那么跨过的部分肯定还是原来直径最长, 答案为 $d - p + w$ 。

1.5 直径和中心

首先观察到若修改,一定修改的是原来直径上的边,否则原来直径不变且最大,肯定不改。

否则考虑修改直径第 i 条边的贡献。有跨过该边的路径和不跨过的,假设该边是 $p \rightarrow w$, 原来直径是 d 。

那么跨过的部分肯定还是原来直径最长, 答案为 $d - p + w$ 。

不跨过的部分是两棵独立的子树, 可以通过序列 dp 得到确定的答案, 不妨设为 y 。

那么修改该边的答案为 $\max(d - p + w, y)$, 显然在 $w \leq y + p - d$ 时取到 y , 其余情况取到 $d - p + w$ 。将直径上的边按它排序, 每次讨论两侧哪边更优即可 (y 最小的答案和 $d - p$ 最小的答案比一下)。

复杂度 $O(n \log n)$, 瓶颈在排序。

1.6 习题

- P1351 [NOIP2014 提高组] 联合权值
- CF1406C Link Cut Centroids
- P5666 [CSP-S2019] 树的重心
- P2680 [NOIP2015 提高组] 运输计划
- P1967 [NOIP2013 提高组] 货车运输
- P1600 [NOIP2016 提高组] 天天爱跑步
- P2491 [SDOI2011] 消防

目录

1. 树基础	2
1.1 定义	3
1.2 树的重心	10
1.3 dfs 序、括号序和欧拉序	13
1.4 LCA	16
1.5 直径和中心	21
1.6 习题	24
2. 最小生成树	25
2.1 定义	26
2.2 Kruskal	27
2.3 Prim	30
2.4 习题	34

2.1 定义

2.1 定义

给定一张 n 个点的连通无向带权图 $G = (V, E)$ ，从中选择 $n - 1$ 条边，使得所有点连通并且边权和最小，得到的树称为**最小生成树**（Minimum Spanning Tree, MST）。

若图不连通，则无法得到覆盖所有点的生成树；此时可以对每个连通块分别求最小生成树，得到**最小生成森林**。

2.1 定义

给定一张 n 个点的连通无向带权图 $G = (V, E)$ ，从中选择 $n - 1$ 条边，使得所有点连通并且边权和最小，得到的树称为**最小生成树**（Minimum Spanning Tree, MST）。

若图不连通，则无法得到覆盖所有点的生成树；此时可以对每个连通块分别求最小生成树，得到**最小生成森林**。

最小生成树常用到两条性质：

(1) **割性质**：对任意一个割，跨过这个割的最小权边一定存在于某棵最小生成树中。

(2) **环性质**：任意一个环上的最大权边一定可以不出现在某棵最小生成树中。

这两条性质都可以用交换法证明：如果当前树里没有想要的边，把它加入后会形成一个环，再删去环上另一条跨过同一割的边即可。

2.2 Kruskal

2.2 Kruskal

- P3366 【模板】最小生成树

给定一个 n 个点 m 条边的无向图，求最小生成树的边权和。若不存在最小生成树，输出 orz。

$1 \leq n \leq 5000$, $1 \leq m \leq 2 \times 10^5$ 。

2.2 Kruskal

- P3366 【模板】最小生成树

给定一个 n 个点 m 条边的无向图，求最小生成树的边权和。若不存在最小生成树，输出 orz。

$1 \leq n \leq 5000$, $1 \leq m \leq 2 \times 10^5$ 。

Kruskal 算法从小到大考虑每条边。

若当前边 (u, v, w) 的两个端点已经连通，那么加入它会产生环，跳过。

否则加入这条边，并合并 u, v 所在的连通块。

2.2 Kruskal

- P3366 【模板】最小生成树

给定一个 n 个点 m 条边的无向图，求最小生成树的边权和。若不存在最小生成树，输出 orz。

$1 \leq n \leq 5000, 1 \leq m \leq 2 \times 10^5$ 。

Kruskal 算法从小到大考虑每条边。

若当前边 (u, v, w) 的两个端点已经连通，那么加入它会产生环，跳过。

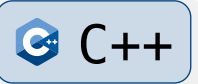
否则加入这条边，并合并 u, v 所在的连通块。

正确性来自割性质：当我们第一次连接两个不同连通块时，当前边是所有仍能连接这两个连通块的边中权值最小的，可以安全加入答案。

实现时使用并查集维护连通块，复杂度为 $O(m \log m)$ ，瓶颈在排序。

2.2 Kruskal

```
1  struct Edge {  
2      int u, v, w;  
3  };  
4  
5  sort(e + 1, e + m + 1, [](Edge a, Edge b) {  
6      return a.w < b.w;  
7  });  
8  
9  long long ans = 0;  
10 int cnt = 0;
```



```
11  for (int i = 1; i <= m; i++) {  
12      int u = e[i].u, v = e[i].v;  
13      if (find(u) == find(v)) continue;  
14      merge(u, v);  
15      ans += e[i].w;  
16      cnt++;  
17  }  
18  
19  if (cnt != n - 1) puts("orz");  
20  else cout << ans << "\n";
```

2.3 Prim

2.3 Prim

Prim 算法从一个点出发，逐步扩大已经被最小生成树覆盖的点集 S 。

每次选择一条从 S 连向 $V - S$ 的最小边，将对应的新点加入 S 。

2.3 Prim

Prim 算法从一个点出发，逐步扩大已经被最小生成树覆盖的点集 S 。

每次选择一条从 S 连向 $V - S$ 的最小边，将对应的新点加入 S 。

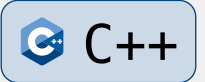
这同样来自割性质：当前 S 与 $V - S$ 构成一个割，跨过这个割的最小边可以安全加入某棵最小生成树。

若使用邻接矩阵并每次扫描所有点，复杂度为 $O(n^2)$ ，适合稠密图。

若使用堆维护候选边，复杂度为 $O(m \log m)$ ，适合稀疏图，写法与 Dijkstra 很像。

2.3 Prim

```
1  priority_queue<pair<int, int>, vector<pair<int,  
    int>>, greater<pair<int, int>>> q;  
2  
3  q.push({0, 1});  
4  long long ans = 0;  
5  int cnt = 0;  
6  
7  while (!q.empty()) {  
8      auto [w, u] = q.top();  
9      q.pop();
```



```
10  if (vis[u]) continue;
11  vis[u] = true;
12  ans += w;
13  cnt++;
14  for (auto [v, c] : adj[u]) {
15      if (!vis[v]) q.push({c, v});
16  }
17 }
18
19 if (cnt != n) puts("orz");
20 else cout << ans << "\n";
```


2.3 Prim

Kruskal 和 Prim 的选择：

- 边数较少、边表输入、需要处理重边时，Kruskal 通常更简单。
- 稠密图或邻接矩阵输入时， $O(n^2)$ Prim 常数较小。
- 如果需要维护“最大生成树”，只需把边权比较方向反过来。

注意最小生成树只要求总边权最小，并不要求任意两点间路径最短；不要与最短路混淆。

2.4 习题

- P2330 [SCOI2005] 繁忙的都市
- P4180 [BJWC2010] 严格次小生成树

Thanks!